

DESCRIPCION ESTRUCTURA 2 ()

Estructura propuesta (estándar)

text

```
mi-proyecto/  
├── .ai/  
│   ├── system.md  
│   ├── rules.md  
│   ├── context.md  
│   └── agents.md  
├── .vscode/  
│   └── settings.json  
├── .env  
├── .gitignore  
├── opencode.json  
└── src/  
    └── ...
```

Función y comportamiento de cada archivo

1) `.ai/system.md` — “Prompt de sistema” del proyecto

Función: define el comportamiento base del asistente para TODO el proyecto: rol, tono, prioridades, límites, formato de respuesta y cómo usar el resto de archivos

`.ai/.`

Cómo se comporta: se carga al iniciar una sesión del agente (OpenCode lo busca como `.ai/system.md`). Actúa como la capa más alta: si algo contradice aquí, normalmente esto manda.

Qué debe contener:

- Identidad/rol del asistente.
- Cómo aplicar reglas (`rules.md`).
- Cuándo y cómo invocar “agentes” (`agents.md`).
- Estilo de respuesta (pasos, formato, nivel de detalle).

- Restricciones (no tocar `.env`, no inventar APIs, etc.).

2) `.ai/rules.md` — Reglas globales obligatorias

Función: reglas innegociables del proyecto (calidad, seguridad, estilo, convenciones, testing, Git workflow, etc.).

Cómo se comporta: el agente debe tratarlas como *hard constraints*. Si una petición del usuario entra en conflicto, el agente debería:

1. avisar del conflicto,
2. proponer alternativa compatible.

Qué debe contener:

- Reglas de seguridad (no exponer secretos, no pegar keys).
- Reglas de cambios (no tocar migraciones sin aprobación, etc.).
- Reglas de estilo (lint, format, nombres de archivos).
- Reglas de output (ej: “siempre incluir comandos para ejecutar tests”).

3) `.ai/context.md` — Contexto técnico del proyecto

Función: “memoria” técnica estable: stack, arquitectura, módulos, dependencias, rutas importantes, esquema de DB, endpoints, decisiones (ADR resumidas), etc.

Cómo se comporta: es información de referencia. No son reglas, sino “hechos del mundo” del proyecto. El agente la usa para no inventar y para orientar el código hacia tu realidad.

Qué debe contener:

- Stack (Node, Python, Next, etc.), versiones.
 - Cómo correr el proyecto.
 - Estructura de carpetas y responsabilidades.
 - DB: tablas, relaciones clave, naming.
 - Servicios externos (Stripe, S3), colas, etc.
-

4) `.ai/agents.md` — Catálogo de agentes especialistas

Función: define “sub-roles” o especialistas (Arquitecto, QA, Security, Data, DevOps, etc.) y cuándo usar cada uno.

Cómo se comporta: no siempre se “ejecutan” como procesos separados; muchas herramientas lo usan como guía para que el mismo modelo cambie de enfoque (“ponte el sombrero de QA”). Sirve para estandarizar cómo se analiza y se responde según el tipo de tarea.

Qué debe contener:

- Lista de agentes, propósito, fortalezas.
- Cuándo invocarlos (criterios).
- Checklist de salida por agente (ej: QA siempre genera casos de prueba).

5) `.vscode/settings.json` — Ajustes del editor (no del agente)

Función: formateo, lint, paths, default formatter, etc.

Cómo se comporta: afecta a VSCode y a tu equipo, no “a la IA” directamente (aunque ayuda a que el código generado encaje con el formato del repo).

6) `.env` — secretos/config local (no se sube)

Función: llaves/API keys/config sensible.

Cómo se comporta: el agente no debe escribirlo ni pedirte que lo pegues completo. En `.ai/rules.md` conviene prohibir explícitamente exponer secretos.

7) `.gitignore`

Función: excluir `.env`, `node_modules`, `builds`, etc.

8) `opencode.json`

Función: configuración específica de OpenCode (modelo, herramientas habilitadas, rutas permitidas, etc. depende de OpenCode).

Cómo se comporta: configura al runtime/herramienta, no al “prompt” textual. Útil para limitar acceso a archivos, activar herramientas, etc.

9) `src/`

Función: código fuente del producto.

Ejemplo “mejor” (contenido sugerido para cada `.ai/* .md`)

`.ai/system.md` (ejemplo)

Markdown

```
# System — mi-proyecto
```

```
## Rol
```

```
Eres el asistente de ingeniería de software para este repositorio. Tu objetivo es ayudar a implementar cambios correctos, seguros y mantenibles.
```

```
## Prioridades
```

- ```
1) Cumplir reglas de .ai/rules.md.
2) Usar el contexto técnico de .ai/context.md.
3) Si falta info, preguntar antes de asumir.
```

```
Estilo de respuesta
```

- ```
- Responde en español.  
- Para cambios de código: entrega (1) plan breve, (2) cambios propuestos por archivo, (3) comandos para probar.  
- No inventes dependencias: si sugieres una librería, justifica y pide confirmación.
```

```
## Agentes
```

```
Usa especialistas cuando aplique (definidos en .ai/agents.md).
```

`.ai/rules.md` (ejemplo)

Markdown

Reglas globales (OBLIGATORIAS)

Seguridad

- No revelar ni solicitar valores completos de ``.env``.
- No incluir secretos en commits ni ejemplos.

Cambios y calidad

- Todo cambio debe incluir cómo probarlo (unit/integration).
- Mantener compatibilidad con Node 20 y TypeScript.
- No introducir nuevas dependencias sin aprobación.

Convenciones

- Nombres: camelCase en JS/TS, kebab-case en archivos.
- Preferir funciones puras y separación por capas (controller/service/repo).

.ai/context.md (ejemplo)

Markdown

Contexto del proyecto

Stack

- Node.js 20, TypeScript
- API: Express
- DB: PostgreSQL
- ORM: Prisma

Estructura

- src/controllers: HTTP handlers
- src/services: lógica de negocio
- src/repositories: acceso a datos
- src/routes: definición de rutas

Ejecutar

- pnpm i
- pnpm dev
- pnpm test

DB (resumen)

- users(id, email, createdAt)
- orders(id, userId, total, status)

Relación: users 1-N orders

.ai/agents.md (ejemplo)

Markdown

Agentes especialistas

Archito (Arquitectura)

Cuándo usar: cambios grandes, refactors, nuevas features.

Checklist de salida:

- impacto en módulos/rutas

- alternativas y trade-offs
- plan de migración si aplica

QA (Calidad)

Cuándo usar: antes de PR, cambios críticos, bugs.

Checklist:

- casos de prueba (happy path + edge cases)
- riesgos de regresión
- comandos para correr tests

Security (Seguridad)

Cuándo usar: auth, permisos, manejo de datos sensibles.

Checklist:

- amenazas comunes
- validación/escape
- recomendaciones OWASP relevantes

¿Falta algún otro archivo?

No es obligatorio, pero en proyectos reales suele valer la pena agregar (opcionales):

1. `.ai/glossary.md` (opcional)
Términos del negocio: “cliente”, “cuenta”, “tenant”, “workspace”, estados, etc.
Evita malentendidos.
2. `.ai/decisions.md` o `docs/adr/` (recomendado)
Decisiones arquitectónicas (por qué Prisma, por qué colas, etc.). Reduce discusiones repetidas.
3. `.ai/playbooks.md` (opcional)
“Runbooks”: cómo desplegar, rotar keys, hacer migraciones, rollback.
4. `CONTRIBUTING.md` (recomendado)
Guía humana (no solo IA): ramas, PRs, lint, commits.
5. `README.md` (imprescindible si no existe)
Cómo correr, testear, desplegar, overview.